

Avancerad Routing i React

Installation, Optimering och Programmatisk Navigering

När en React-applikation växer och får många olika vyer räcker det inte med en enkel routing-lista. Om vi inte strukturerar appen rätt kommer den att bli långsam att ladda, och vi kommer tvingas skriva onödigt mycket duplicerad kod. Genom att använda moderna tekniker i React Router kan vi optimera prestandan, återanvända gemensamma layouter och styra användarens navigering helt via kod.

0. Innan vi börjar: Installation

Till skillnad från ramverk som Angular eller Next.js, kommer React inte med något inbyggt system för att hantera URL:er och flera sidor. React är i grunden bara ett bibliotek för att bygga användargränssnitt. För att få tillgång till alla komponenter vi ska använda måste vi först installera paketet via terminalen.

Kör följande kommando i projektets rotmapp:

```
npm install react-router-dom
```

(Notera att paketet heter **react-router-dom** och inte bara "react-router". Tillägget "-dom" innebär att vi arbetar specifikt för webben och webbläsarens gränssnitt).

1. Prestanda: Lazy Loading och Suspense

I en standard-applikation bakas all JavaScript-kod – för alla dina sidor och komponenter – in i en och samma stora fil (en så kallad *bundle*) när appen byggs. Detta innebär att en

besökare som bara vill se din "Startsida" tvingas ladda ner koden för "Inställningar", "Admin-panelen" och "Profilsidan" innan sidan ens kan visas. Detta leder till tröga laddningstider.

Lösningen kallas **Code Splitting** och **Lazy Loading**. Genom att använda funktionen `lazy()` instruerar vi React att stycka upp koden. Appen hämtar nu bara koden för till exempel "Dashboard" från servern i exakt det ögonblick användaren faktiskt klickar på den länken.

Eftersom den begärda koden inte finns i webbläsaren direkt, tar det några millisekunder att hämta filen. Under denna korta fördröjning måste React veta vad som ska visas på skärmen för att inte krascha. Det är här **Suspense** kommer in. Den fungerar som ett säkerhetsnät som automatiskt visar en fallback (till exempel en laddningssnurra eller text) tills den nya sidan är färdigladdad.

```
import { lazy, Suspense, type JSX } from "react";
import { Routes, Route } from "react-router-dom";

// Lazy-Loaded pages - filerna hämtas från servern först när de behövs
const Home = lazy(() => import("./pages/Home"));
const About = lazy(() => import("./pages/About"));
const Dashboard = lazy(() => import("./pages/Dashboard"));

export default function App() {
  return (
    {/* Suspense omsluter hela vårt routing-träd */}
    <Suspense fallback=<div>Laddar sidan...</div>>
      <Routes>
        {/* Rutt-konfigurationen kommer här... */}
      </Routes>
    </Suspense>
  );
}
```

2. Nästlade Rutter (Nested Routes)

Det är sällan en webbsida byter ut hela gränssnittet när man byter sida. Oftast har man en toppmeny (navbar) och en sidfot (footer) som alltid ligger kvar, medan bara innehållet i

mitten byts ut. Ett nybörjarmisstag är att importera komponenten `Navbar` högst upp i varje enskild fil (i `Home.tsx`, `About.tsx`, osv). Detta är ineffektivt och svårt att underhålla.

Ett mycket bättre mönster är **Nästlade rutter**. Vi skapar en huvudkomponent (ofta kallad `Layout`) som fungerar som ett skal för hela applikationen. Inuti vår routing talar vi sedan om att specifika sidor ska laddas *inuti* detta skal. För att hantera rutter som saknar ett specifikt sökvägsnamn (som startsidan) använder vi nyckelordet `index`, vilket markerar den komponenten som standardval för föräldrarutten.

App.tsx (Rutt-strukturen)

```
<Routes>
  {/* Huvud-layouten omsluter allt nedanför. */}
  <Route path="/" element={<Layout />}>

    {/* "index" betyder att Home visas som standard när vi är exakt på "/" */}
    <Route index element={<Home />} />
    <Route path="about" element={<About />} />

    {/* Dynamisk rutt: :id innebär att URL:en kan vara /profile/1 eller /pr */}
    <Route path="profile/:id" element={<Profile />} />

    {/* Fallback: Fångar upp alla okända URL:er och visar en 404-sida */}
    <Route path="*" element={<NotFound />} />

  </Route>
</Routes>
```

Layout.tsx (Mallen med Outlet)

I själva Layout-filen använder vi komponenten `Outlet` från React Router. Man kan tänka på `Outlet` som ett tomt fönster eller ett dynamiskt platshållar-hål. När React Router ser att URL:en matchar `"/about"`, tar den din `About`-komponent och skjuter automatiskt in den exakt där `Outlet` är placerad i skalet.

```
import { NavLink, Outlet } from "react-router-dom";

export default function Layout() {
  return (
    <>
      <nav style={{ display: "flex", gap: "1rem" }}>
        {/* 'end' ser till att Hem-länken bara är "aktiv" när vi är EXAKT p
        <NavLink to="/" end>Hem</NavLink>
        <NavLink to="/about">About</NavLink>
        <NavLink to="/profile/42">Profil 42</NavLink>
      </nav>

      <main style={{ padding: "20px" }}>
        {/* Det dynamiska innehållet renderas här */}
        <Outlet />
      </main>
    </>
  );
}
```

3. NavLink vs Link (Navigering och styling)

I en Single Page Application (SPA) får vi aldrig använda vanliga HTML-länkar (`<a href="/about">`) eftersom det tvingar webbläsaren att ladda om hela sidan och därmed rensa hela Reacts tillstånd (state). Istället använder vi `Link` som smidigt byter ut komponenterna i minnet.

I vår Layout-fil ovan använder vi dock en specialversion: `NavLink`. Skillnaden är att `NavLink` är medveten om den aktuella URL:en. Om användaren befinner sig på `"/about"`, kommer `NavLink` automatiskt att lägga till en CSS-klass som heter `.active` på just den länken. Detta gör det extremt enkelt att visuellt markera i menyn var besökaren befinner sig.

Viktigt om "end": Eftersom startsidans länk leder till `"/`, och alla andra rutter (som `"/about"`) *börjar* med ett snedstreck, skulle startsidans länk annars alltid tro att den är aktiv. Genom att lägga till egenskapen `end` på startsidans `NavLink` säger vi: *"Bli bara aktiv om URL:en slutar exakt här."*

4. Programmatisk Navigering (`useNavigate`)

Ibland sker navigering inte för att användaren klickar på en specifik länk i en meny, utan som en konsekvens av en handling (programmatisk navigering). Till exempel:

- En användare fyller i ett formulär för att skapa en ny produkt och klickar på "Spara". När anropet till API:et är klart, vill du automatiskt skicka dem till produktlistan.
- En användare loggar in och skickas vidare till sin instrumentpanel (Dashboard).
- En knapp för att avbryta ett flöde och gå ett steg tillbaka i historiken.

För att lösa detta använder vi hooken `useNavigate`. Den ger oss en funktion som vi kan anropa var som helst inuti vår logik för att styra webbläsarens URL.

About.tsx (Knapp-navigering)

```
import { useNavigate } from "react-router-dom";

export default function About() {
  const navigate = useNavigate();

  const handleLogin = () => {
    // Låtsas utföra någon logik, därefter byter vi sida direkt:
    navigate("/dashboard");
  };

  return (
    <>
      <h1>About</h1>
      {/* navigate("/") skickar användaren till startsidan */}
      <button onClick={() => navigate("/")}>Tillbaka hem</button>

      {/* Du kan också använda negativa tal för att gå bakåt i webbhistorik */}
      <button onClick={() => navigate(-1)}>Gå föregående sida</button>
    </>
  );
}
```

5. Finns det alternativ till React Router?

React Router har länge varit den absoluta standarden för routing i React-applikationer. Men de senaste åren har webbutvecklingen utvecklats snabbt, och idag finns det flera starka alternativ beroende på vad man bygger:

- **TanStack Router:** Ett av de snabbast växande biblioteken just nu. Det stora dragplåstret med TanStack Router är att det är byggt från grunden med 100% fokus på TypeScript och typsäkerhet (Type Safety). Det gör att du får autokomplettering och varningar från redigeraren om du försöker skicka användaren till en URL som inte existerar.
- **Next.js (App Router / Pages Router):** Om du bygger en fullskalig webbapplikation med React rekommenderas ofta ramverket Next.js. I Next.js behöver du inte installera eller skriva koden för React Router. Istället använder de *filsystemsbaserad routing*. Det betyder att varje fil du skapar inuti en specifik mapp (t.ex. `app/about/page.tsx`) automatiskt blir en rutt (`/about`) på webbplatsen.
- **Wouter:** Om du bygger ett väldigt litet projekt och tycker att React Router är för stort och klumpigt att installera, är Wouter ett minimalistiskt alternativ. Det tar upp extremt lite plats och fokuserar bara på de mest grundläggande funktionerna för navigering.